

Module 03

Redis + arq (Job Queues)

Background tasks, cron scheduling, and deferred jobs

ContentAutomation2 — Backend Learning Series
Zero Prerequisites → Production-Ready Code

1. What is a Job Queue and Why Do You Need One?

Some tasks take too long to run inside an HTTP request. If a user hits POST /run-research and the research takes 10 minutes, the HTTP request would time out. Instead, you put the task into a **queue** and return immediately. A separate **worker** process picks it up and runs it in the background.

Real-world uses: sending emails, processing images, scraping websites, running ML models, syncing data, generating reports.

2. Redis — The Queue Backbone

Redis is an in-memory key-value store. It's extremely fast (microsecond operations) and supports data structures like lists, sets, hashes, and sorted sets. Job queues use Redis lists as queues: the API **pushes** job data onto a list; the worker **pops** from the list to get the next job.

```
# Install Redis on Windows (easiest: use WSL or Docker)
# docker run -d -p 6379:6379 redis:alpine

# Install Python client
pip install redis

import redis

r = redis.Redis(host="localhost", port=6379, decode_responses=True)

# Basic operations
r.set("name", "Himanshu")          # SET key value
val = r.get("name")                 # GET key → "Himanshu"
r.lpush("my_queue", "job1")         # push to list (left end)
r.lpush("my_queue", "job2")
job = r.brpop("my_queue", timeout=5) # blocking pop (right end)
print(job)                          # ("my_queue", "job1")
```

3. arq — Async Job Queue for Python

arq (async Redis Queue) is a Python library that wraps Redis and provides a clean API for defining tasks, enqueueing them, and running a worker that processes them. It is built for async Python (asyncio).

```
pip install arq

# tasks.py — define your task functions
async def send_email(ctx: dict, to: str, subject: str) -> dict:
    # ctx is injected by arq — holds shared resources
    print(f"Sending email to {to}: {subject}")
    await asyncio.sleep(0.5) # simulate email sending
    return {"status": "sent", "to": to}

async def resize_image(ctx: dict, image_path: str) -> dict:
    print(f"Resizing {image_path}")
    return {"resized": True}
```

4. WorkerSettings — Registering Tasks

```
# worker.py
from arq.connections import RedisSettings
from tasks import send_email, resize_image

class WorkerSettings:
    # List all your task functions here
    functions = [send_email, resize_image]

    # Where Redis is running
    redis_settings = RedisSettings(host="localhost", port=6379)

    # Max jobs to run simultaneously
    max_jobs = 10

    # Max seconds a job can run before being killed
    job_timeout = 300

    # Run at startup / shutdown (optional)
    async def on_startup(ctx):
        print("Worker starting up...")
        ctx["db"] = await connect_db() # shared across all tasks

    async def on_shutdown(ctx):
        await ctx["db"].close()

# Run the worker:
# python -m arq worker.WorkerSettings
```

5. Enqueueing Jobs — From Your API or Anywhere

```
import arq
from worker import WorkerSettings

async def main():
    # Create a connection pool to Redis
    pool = await arq.create_pool(WorkerSettings.redis_settings)

    # Enqueue a job – runs as soon as the worker picks it up
    job = await pool.enqueue_job("send_email", to="user@example.com", subject="Hello")
    print(f"Enqueued job: {job.job_id}")

    # Enqueue with a delay (runs after 1 hour)
    from datetime import timedelta
    job2 = await pool.enqueue_job(
        "resize_image",
        image_path="/uploads/photo.jpg",
        _defer_by=timedelta(hours=1),
    )

    await pool.close()
```

Note: In *ContentAutomation2*, the FastAPI app creates an arq pool at startup (lifespan) and stores it in `app.state.arq_pool` so all endpoints can enqueue jobs.

6. Cron Jobs — Scheduled Recurring Tasks

```
from arq import cron
from tasks import send_email, cleanup_db

class WorkerSettings:
    functions = [send_email, cleanup_db]
    redis_settings = RedisSettings()

    cron_jobs = [
        # Run cleanup_db every day at 3 AM
        cron(cleanup_db, hour=3, minute=0),

        # Run send_email every hour at minute 0
        cron(send_email, to="admin@example.com",
            subject="Hourly report", minute=0),

        # Run every 15 minutes
        cron(send_email, to="admin@example.com",
            subject="15-min check", minute={0, 15, 30, 45}),
    ]
```

Note: ContentAutomation2 runs the research agent daily at 6 AM IST (00:30 UTC) and the publishing agent every 15 minutes via cron_jobs.

7. Job Chaining — Triggering the Next Job on Completion

One job can enqueue another job when it finishes. This creates a pipeline: research → scoring → creation, all triggered automatically.

```
async def research_agent_task(ctx: dict) -> dict:
    # ... do research work ...

    # When done, automatically kick off the scoring agent
    arq_pool = ctx.get("redis") # arq injects the pool as "redis" in ctx
    if arq_pool is not None:
        await arq_pool.enqueue_job("scoring_agent_task")

    return {"status": "done", "processed": 42}
```

8. The ctx Dict — Shared Resources

Every task receives a ctx dict as its first argument. arq populates it with shared resources from on_startup. You can store database connections, settings, or any object there.

```
class WorkerSettings:
    async def on_startup(ctx: dict):
        from myapp.db import get_db_client
        from myapp.config import Settings
        ctx["db"] = get_db_client() # shared DB client
        ctx["settings"] = Settings() # app config
        ctx["redis"] # already injected by arq – the arq connection pool
    async def my_task(ctx: dict, item_id: str) -> dict:
```

```
db          = ctx["db"]
settings = ctx["settings"]
pool        = ctx["redis"]    # can enqueue more jobs
...
```

9. How ContentAutomation2 Uses arq

The project has 6 task functions: research, scoring, creation, publishing, analytics, and login_site. The worker is started with `python -m arq app.queue.worker.WorkerSettings`. The FastAPI app enqueues jobs via `app.state.arq_pool`.

10. Practice Exercises

- Exercise 1: Start Redis with Docker: `docker run -d -p 6379:6379 redis:alpine`
- Exercise 2: Write a task `add_numbers(ctx, a, b)` that returns `a+b` after 1 second.
- Exercise 3: Write `WorkerSettings` that registers `add_numbers`.
- Exercise 4: Write a script that enqueues `add_numbers(3, 7)` and prints the `job_id`.
- Exercise 5: Start the worker and watch it process the job.
- Exercise 6: Add a cron job that runs `add_numbers(1, 1)` every minute.
- Exercise 7: Chain: make `add_numbers` enqueue a `log_result` task when it finishes.